

Comprehensive Strategic and Architectural Report: WeMakeDevs × AWS Environmental Hacks (Bharat Builds Tour)

Event Architecture, Regulatory Guidelines, and Strategic Objectives

The Environmental Hacks hackathon represents the second stop of the six-city hybrid Bharat Builds Tour, organized by WeMakeDevs in direct collaboration with the Amazon Web Services (AWS) Builder Center¹. Operating within a cumulative tour prize pool exceeding ₹1 crore, this national four-day technical sprint focuses on converting raw atmospheric, hydrological, and municipal waste telemetry into actionable operational interventions¹. The hackathon takes place virtually across India from Thursday, October 8 through Sunday, October 11, 2026, accompanied by an optional, limited-capacity physical Build Day on Saturday, October 10 at Delhi Technological University (DTU)¹.

The event architecture utilizes a hybrid execution pipeline designed to decouple local event participation from project evaluation¹. While attending the physical Build Day at DTU provides direct access to AWS solutions architects, interactive workshops, and project feedback from industry mentors, on-site presence does not influence scoring¹. All projects—whether developed entirely remotely or on-site—are evaluated synchronously by an centralized judging panel using identical evaluation criteria¹.

Phase / Event	Operating Window	Operational Environment	Compliance Requirements & Administrative Constraints
Registration & Verification	Pre-event window through Oct 8, 2026	WeMakeDevs Portal & AWS Builder Center	Requires linking a verified student profile via SheerID to unlock profile rewards and Amazon recruitment eligibility ¹ .

Sprint Window	Oct 8, 2026 – Oct 11, 2026	Virtual (Nationwide across India)	All functional code must be authored strictly within the competition clock ¹ . Pre-existing codebases are prohibited ¹ .
DTU Physical Build Day	Oct 10, 2026 (8:00 AM – 8:00 PM IST)	Delhi Technological University (DTU), Delhi	Optional in-person seat; requires secondary registration via Luma ¹ . Attendance does not impact project scoring ¹ .
Submission Deadline	Oct 11, 2026 (8:00 AM IST strictly)	WeMakeDevs Form Portal	Form closes automatically at the deadline ¹ . Requires public GitHub repository, 3-minute YouTube video, and architecture write-up ¹ .
Panel Review & Dispatch	Post-submission evaluation window	Virtual Evaluation Panel	Evaluated on video demonstration, execution stability, real-world utility, and AWS implementation ¹ .

The competition is partitioned into three domain-specific problem tracks¹. Rather than grouping submissions into a single national pool, each track operates with its own dedicated prize allocation, ensuring domain-specific evaluation¹.

Track Identifier	Atmospheric & Ecological Scope	Domain Focus & Target Vectors
Track 01: Air	Atmospheric monitoring,	AQI prediction, stubble

	exposure mitigation, and urban air quality management ¹ .	burning detection, indoor air purification control, localized pollution alerting, and school safety directives ¹ .
Track 02: Heat and Water	Hydrological extremes, urban microclimate adaptation, and water distribution infrastructure ¹ .	Heatwave early warning, monsoon waterlogging navigation, drought forecasting, tanker logistics, distribution leak detection, and groundwater management ¹ .
Track 03: Waste and Energy	Circular material flows, resource recovery systems, and clean energy optimization ¹ .	Automated waste segregation, e-waste lifecycle tracking, informal recycler integration, rooftop solar efficiency, EV behavioral nudges, and transit routing ¹ .

The financial and institutional incentive structure combines direct monetary awards, cloud credit grants, hardware rewards, and recruitment opportunities¹. The recruitment pipeline with Amazon's University Talent Acquisition Team operates independently of the track prize outcomes¹. A project that does not secure a cash or track prize remains fully eligible for fast-track interview shortlisting, provided the project exhibits structural resilience, production readiness, and architectural utility¹.

Reward / Identifier	Recipient Allocation	Direct Monetary & Institutional Value
Track Winner	1 Team per Track (3 Teams total)	₹2,00,000 cash disbursed directly + \$2,00,000 AWS Cloud Credits per team ¹ .
Hackathon Runners-Up	4 Teams (Selected across all tracks)	\$1,000 AWS Cloud Credits per team ¹ .
Top Technical Blogs	5 Individual Authors	Apple AirPods 5 (Awarded for architectural posts)

		published on AWS Builder Center) ¹ .
Amazon Fast-Track Interviews	Up to 10 Students (Across all tracks)	Direct fast-track interview loop for 6-month internships or full-time SDE roles, bypassing preliminary screening rounds ¹ .
Student Verification Perks	All SheerID Verified Builders	\$579 total value: \$449 Skill Builder Premium (1 year), \$100 AWS Certification Voucher, \$30 AWS direct credits ² .

Participation guidelines strictly govern account creation, team structures, and code authenticity¹. Entry is open to undergraduate and postgraduate university students in India aged 18 or older¹. Teams may range from 1 to 4 individuals, with cross-university and geographically distributed compositions fully permitted¹. Every participant must maintain a registered WeMakeDevs account and an AWS Builder Center profile verified via SheerID¹. If a student's SheerID verification remains pending during the event window, the student is still permitted to submit projects for panel evaluation; however, final interview shortlisting and profile reward claims remain locked until verification clears¹.

All code must be authored during the four-day hackathon window¹. Repurposing pre-existing repositories, submitting legacy software, or forging commit timestamps results in immediate team disqualification¹. Teams are permitted to utilize public open-source libraries, frameworks, starter templates, and AI coding assistants, provided all external dependencies are disclosed in the project documentation¹. AWS cloud deployment requires an active AWS account, which can be created using standard debit or RuPay cards with a nominal authorization charge of approximately ₹2¹. Every new account receives up to \$200 in AWS Free Tier credits, which is sufficient to deploy serverless and agentic architectures without incurring out-of-pocket costs¹. Teams requiring capacity beyond the standard free tier allocations may request supplementary credits directly through the event organizers¹.

Synthesis of Previous Winners and Competitive Benchmarking

Evaluating winning architectures from Event 01 of the tour (*First Commit*) reveals the technical standards and structural paradigms favored by the evaluation panel⁶. First Commit brought together over 13,000 registered developers competing across general software domains, setting a baseline for system safety, execution stability, and user experience⁶.

Project Name	Award Category	Target Problem Domain	Primary AWS Architectural Components	Critical Winning Factors
Suraksha (StarBugs)	1st Place Overall (Ship It)	Elder Care & Emergency Response	Agentic AI framework, real-time alerting, AWS Cloud deployment ⁶ .	Proactive intervention mechanics for vulnerable demographics; smooth end-to-end cloud integration ⁶ .
Beacon Night Shift (CoffeeAndCode)	2nd Place Overall (Build It)	Infrastructure & On-Call Automation	Amazon Bedrock, AWS Step Functions, Amazon Transcribe, Amazon Polly, DynamoDB, AWS Lambda, CloudWatch ⁶ .	Human-in-the-loop voice verification, temporal "Sleep Contracts", deterministic safety constraints, clean build testing ⁷ .
CampusEvac (3 BUILDS)	3rd Place Overall (Best UI)	Emergency Civic Evacuation	Spatial mapping, dynamic routing, frontend rendering engine ⁶ .	Exceptional visual hierarchy, intuitive navigation under simulated high-stress civic emergencies ⁶ .
FloodRoute (HeisenBergs)	High-Ranking Finalist	Urban Hydrology & Navigation	Risk-aware routing algorithms, spatial flooding overlays ⁸ .	Actionable routing logic during severe urban waterlogging

				and flash flood events ⁸ .
--	--	--	--	---------------------------------------

A technical analysis of *Beacon Night Shift*—the winner of the Build It track at First Commit—provides an operational blueprint for hackathon project execution⁶. Built to resolve developer burnout caused by overnight production pages, the system acts as a voice-interactive agent that diagnoses infrastructure alerts, proposes remediations, and awaits explicit human authorization⁷.

The system's architecture relies on Amazon Bedrock for reasoning, orchestrated alongside AWS Step Functions to manage execution loops⁷. Voice interactions are processed via Amazon Transcribe and Amazon Polly, while state persistence, serverless computation, and telemetry monitoring are handled by Amazon DynamoDB, AWS Lambda, and Amazon CloudWatch respectively⁷.

To prevent autonomous AI agents from making unintended system modifications, the developer built strict deterministic bounds into the software loop⁷. The agent is restricted to executing a small whitelist of predefined commands and never assumes consent without explicit spoken authorization⁷. Once an engineer approves a specific fix, the system can issue a temporal, problem-specific "Sleep Contract"⁷. If the exact issue recurs overnight, the system handles the remediation silently, logging summaries for review the following morning⁷. Before submitting, the developer performed clean-environment deployment testing, setting up the repository from scratch on an isolated environment⁷. This process uncovered two deployment errors that were missed during local development, allowing them to be corrected prior to evaluation⁷. Finally, the submission video prioritized problem validation, showcasing the live agent resolving an infrastructure incident and executing an automated Sleep Contract within a three-minute demo⁷.

Diagnostic Analysis of Participant Pitfalls and Organizer Expectations

Data from previous environment-focused hackathons highlights a recurring issue in project design: the **Passive Dashboard Anti-Pattern**². Teams frequently spend the sprint aggregating public sensor telemetry (such as OpenAQ or CPCB feeds), plotting latitudinal and longitudinal coordinates on a map, and highlighting high-pollution or flooded areas in red². Organizers and judges penalize this approach because visual monitoring alone does not change human behavior². Citizens living in affected regions are typically already aware of localized pollution or waterlogging; displaying the data visually adds minimal operational value².

To stand out, submissions must transition from descriptive telemetry to prescriptive decision support². The project must process environmental inputs through a reasoning or policy engine to deliver explicit operational directives to a target user². A project's user persona must be defined before selecting technology stacks².

For example, a project targeting school administrators should process predictive AQI trends to automatically issue facility directives—such as shifting morning assemblies indoors 12 hours

before peak particulate events occur². A system built for residential housing societies should analyze regional water supply metrics to schedule tanker deliveries before local shortages cause price spikes². Similarly, a municipal ward office tool should generate optimized collection routes for waste management fleets based on real-time fill-level telemetry².

Attempting to solve broad global challenges within four days often results in incomplete software¹. Focusing on a single university campus, housing society, or urban ward allows teams to deliver stable, functional features within the competition timeframe¹. In addition, teams must maintain data source transparency². Declarations regarding data sources, update frequencies, and spatial resolution must be explicitly stated in the project documentation². For instance, combining hourly ground-level sensor data with twice-daily satellite passes (e.g., Sentinel-2) requires distinct handling; claiming real-time capabilities on low-frequency satellite data weakens project credibility during technical review².

Evaluation Criteria	Weight	Technical & Functional Expectation	Key Strategic Objective
Idea and Impact	30%	Solves a real environmental problem with demonstrable end-user impact ¹ .	Prioritize actionable operational outcomes over passive data visualization ¹ .
Built on AWS	25%	Mandatory usage of AWS cloud services or AWS open-source projects ¹ .	Demonstrate deep integration with AWS tools (e.g., Strands Agents SDK, Bedrock, SAM, Step Functions) ¹ .
Execution	20%	System must execute reliably; core features must run without crashing ¹ .	Focus on functional stability over unverified feature complexity ¹ .
Design & Usability	15%	Interface must be intuitive for non-technical users ¹ .	Ensure external stakeholders can navigate the system without instructions ¹ .

Demo Video	10%	Concise 3-minute video uploaded to YouTube ¹ .	Demonstrate system functionality, user workflows, and AWS architecture ¹ .
------------	-----	---	---

Technical Execution Framework and Architecture Blueprints

Participants can choose between two development pathways to satisfy the AWS integration mandate, both of which are evaluated equally by the judging panel¹.

Pathway Identifier	Environment & Tech Stack	Target Use Case & Operational Profile
Build It <i>(Local / Open Source)</i>	Strands Agents SDK, Cedar Policy Engine, LocalStack, SAM CLI, Finch, EKS Distro, OpenSearch, Firecracker ¹ .	Local AI agent orchestration, deterministic policy enforcement, or offline data processing without active cloud costs ¹ .
Ship It <i>(Cloud Deployed)</i>	Amazon Bedrock, SageMaker AI, AWS Lambda, Step Functions, DynamoDB, S3, ECS/EKS, CloudFront, Cognito, EventBridge ¹ .	Live web applications, public API endpoints, serverless event pipelines, or production integrations ¹ .

Environment hackathons often face delays during initial data ingestion when teams attempt to download large satellite datasets locally². The **Registry of Open Data on AWS** addresses this challenge by hosting public environmental datasets directly in Amazon S3 buckets within the cloud environment².

These repositories include Sentinel-2 and Landsat-8 optical imagery for tracking urban heat islands, surface water coverage, and vegetation index changes². NOAA GOES provides continuous geostationary atmospheric data, while NASA POWER offers solar irradiance and meteorological metrics useful for optimizing solar installations and agricultural schedules². Querying these datasets directly via AWS SDKs allows teams to bypass local data ingestion pipelines entirely².

For AI-driven applications, AWS provides the **Strands Agents SDK**—an open-source framework in Python and TypeScript designed for building production-ready AI agents¹⁰. The framework works natively with Amazon Bedrock (including Claude 3.5 Sonnet and Haiku),

Anthropic, OpenAI, and local Ollama instances⁹. It implements the Model Context Protocol (MCP) for tool integration and system context, while providing lifecycle hooks that allow developers to intercept agent actions before and after tool execution⁹. Context and memory management are handled via sliding windows, summarization managers, and vector store integrations such as Amazon OpenSearch⁹.

```
Python
from strands import Agent
from strands.models import BedrockModel
from strands.tools import DuckDuckGoSearch

# Initialize Bedrock provider with Anthropic Claude 3.5 Haiku
bedrock_model = BedrockModel(
    model_id="anthropic.claude-3-5-haiku-20241022-v1:0"
    region_name="us-east-1"
)

# Custom tool for dynamic facility control
def trigger_hvac_purification_system(zone_id: str, intensity: str) -> str:
    """Issues direct operational commands to campus HVAC purification units."""
    return f"HVAC filtration in zone {zone_id} set to {intensity} intensity."

# Construct the agent with model provider and tool sets
agent = Agent(
    model=bedrock_model
    tools=[trigger_hvac_purification_system, DuckDuckGoSearch()]
    system_prompt="You are an environmental actuation agent. Evaluate AQI inputs and issue HVAC commands."
)

# Execute evaluation loop
response = agent("DTU Campus Station 04 reports PM2.5 at 340 ug/m3. Execute mitigation.")
print(response)
```

To enforce safety guardrails on AI agents operating in civic or industrial contexts, teams can use **Cedar**—AWS's open-source policy language¹. Cedar decouples access control policies from application logic, ensuring agents operate within defined parameters¹¹.

```
Code snippet
ceda policy enforcing access controls on environmental actuation agents
permit
principal == Agent:"AOI_Mitigation_Agent"
action == Action:"TriggerPurification"
resource == BuildingZone:"DTU_Auditorium"
|
when
resource.air_quality_index > 200 &&
principal.authorization_level == "Automated_Safety_Level_1"
|

forbid
principal
action == Action:"TriggerPurification"
resource
|
when
resource.emergency_override == true
|
```

Submission Blueprint and Developer Execution Roadmap

To maintain steady progress across the four-day event, teams should align their development workflow with the following timeline:

Development Phase	Target Operating Window	Milestone Deliverables & Technical Execution Scope
Phase 01: Setup & Ideation	Day 01 (Oct 8, 2026)	Select problem domain, define target user persona, configure AWS environment, and complete SheerID student verification ¹ .

Phase 02: Core Engineering	Day 02 (Oct 9, 2026)	Build data ingestion pipelines, integrate Strands Agents SDK or serverless cloud services, and write policy rules ¹ .
Phase 03: Testing & Refinement	Day 03 (Oct 10, 2026)	Refine user interface, perform edge-case handling, and test repository deployment on a clean machine environment ² .
Phase 04: Submission Delivery	Day 04 (Oct 11, 2026)	Record 3-minute video demonstration, publish technical blog post on AWS Builder Center, and finalize GitHub documentation ¹ .

Project repositories must include a comprehensive README.md detailing the single-sentence user persona, target problem domain, detailed AWS architecture diagram, setup instructions, and credit disclosures for third-party libraries¹. An open-source license (such as Apache-2.0 or MIT) should be included alongside a clean setup script (setup.sh, Makefile, or docker-compose.yml) verified through isolated machine testing¹. Commit histories must authentically reflect active development between October 8 and October 11¹. Because evaluation relies entirely on submitted materials without live presentation rounds, the 3-minute video demonstration is critical to project scoring¹.

Video Segment	Target Duration	Required Narrative & Demonstration Focus
01. Problem & Persona	00:00 – 00:30	Define the localized environmental issue and introduce the target user persona ² .
02. System Demonstration	00:30 – 01:45	Show real-time software execution, demonstrating inputs, processing logic, and actionable outputs ¹ .

03. Architecture Review	01:45 – 02:30	Explain the underlying AWS infrastructure (e.g., Bedrock, Step Functions, S3, or Strands SDK) ¹ .
04. Impact & Scalability	02:30 – 03:00	Summarize deployment feasibility, data freshness, and future development steps ² .

Before submitting, teams should verify video permissions by opening the YouTube link in an incognito browser window¹. Narrations should clearly explain the workflow and user impact while showcasing a working implementation¹.

Publishing an architectural write-up on the AWS Builder Center offers teams an opportunity to compete for additional prizes, including Apple AirPods 5¹. The post should cover three key areas: *The Environmental Problem*, *The AWS Architecture Stack*, and *Engineering Challenges Overcome*¹. Including system diagrams, code snippets (such as Cedar policies or Strands hooks), and execution screenshots helps demonstrate technical depth⁷. Once published, the article URL must be linked directly in the hackathon submission form¹.

Strategic Recommendations and Final Synthesis

To maximize competitive performance at the WeMakeDevs × AWS Environmental Hacks, teams must move beyond simple data monitoring and focus on building prescriptive decision-support systems². The following core principles summarize the recommended approach:

First, build workflows that execute specific interventions rather than displaying passive data visualizations². Projects should automate operational responses for a clearly defined user persona—such as a school administrator, housing society board, or municipal officer—operating within a hyper-local setting².

Second, leverage the AWS ecosystem effectively¹. Tools such as the Strands Agents SDK, Cedar policy engine, Amazon Bedrock, and AWS Step Functions allow teams to create resilient architectures while utilizing the AWS Free Tier and Registry of Open Data¹.

Third, prioritize execution stability¹. Conducting clean-environment installation tests ensures the software deploys reliably for judges⁷. Presenting these features clearly within a high-density, 3-minute video demonstration ensures the system's value is effectively communicated to the evaluation panel¹.

Adhering to these engineering and presentation standards enables teams to deliver functional solutions that satisfy both competition criteria and professional recruitment standards¹.

Works cited

1. Environmental Hacks Bharat Builds.txt
2. Announcing Environmental Hacks | WeMakeDevs,

- <https://www.wemakedevs.org/blogs/announcing-environmental-hacks>
3. Bharat Builds Tour - WeMakeDevs, <https://www.wemakedevs.org/aws>
 4. Environmental Hacks | Bharat Builds Tour - WeMakeDevs, <https://www.wemakedevs.org/aws/env>
 5. First Commit | Bharat Builds Tour - WeMakeDevs, <https://www.wemakedevs.org/aws/first-commit>
 6. Winners | First Commit - WeMakeDevs, <https://www.wemakedevs.org/aws/first-commit/projects>
 7. My First Commit hackathon experience: building Beacon Night Shift, <https://builder.aws.com/content/3K5kB6E1Rv7Nm6uCWpRuDIYDSq5/my-first-commit-hackathon-experience-building-beacon-night-shift>
 8. All projects | First Commit | WeMakeDevs, <https://www.wemakedevs.org/aws/first-commit/projects/all>
 9. Getting Started with Strands Agents - Complete Learning Path - GitHub, <https://github.com/aws-samples/sample-getting-started-with-strands-agents-course>
 10. strands-agents/harness-sdk: Build an agent harness and ... - GitHub, <https://github.com/strands-agents/harness-sdk>
 11. How to Build a Real-Data AI Project on the AWS Open Source Stack, <https://www.wemakedevs.org/blogs/how-to-build-a-real-data-ai-project-on-the-aws-open-source-stack>
 12. Blog | WeMakeDevs, <https://www.wemakedevs.org/blogs>
 13. strands-agents - GitHub, <https://github.com/strands-agents>
 14. aws-samples/sample-Advanced-Strands-Agents-with-MCP - GitHub, <https://github.com/aws-samples/sample-Advanced-Strands-Agents-with-MCP>